

HLD Case Study 1: Real-Time Messaging Platform (WhatsApp / Discord)

Key Focus Areas: Real-time WebSockets, message delivery guarantees (sent, delivered, read), distributed connection servers, and offline message synchronization.

1. Problem Statement & Requirements

Design a global real-time chat platform supporting 1-on-1 and group messaging with delivery receipts.

Functional Requirements:

1. Low-latency 1-on-1 and group messaging ($< 100\text{ms}$ delivery).
2. Message status tracking: `Sent` $\rightarrow$ `Delivered` $\rightarrow$ `Read`.
3. Support offline users (queue messages until user reconnects).
4. Media sharing (images/videos up to 100MB).

Non-Functional Requirements:

1. High availability (99.99%).
2. Low latency worldwide.
3. End-to-End Encryption (E2EE) support.

2. Back-of-the-Envelope Estimations

- **DAU:** 500 Million active users.
- **Message Volume:** 40 messages per user/day $\rightarrow$ 20 Billion messages/day.
- **Write QPS:** $\frac{20 \times 10^9}{10^5} \approx 200,000\text{ QPS}$ (Peak = 400,000 QPS).
- **Storage per day:** Average message size 100 bytes $\rightarrow$ 20 Billion $\times$ 100 bytes = 2 TB/day text storage.
- **Concurrent Connections:** ~ 50 Million simultaneous WebSocket connections.

3. High-Level Architecture Diagram

flowchart TD

```

ClientA["Sender Client A"] -->|1. WebSocket Connection| WS_Gateway["WebSocket Gateway"]
WS_Gateway --> SessionSvc["Session Registry (Redis Cluster)"]
User --> Connected_WS
WS_Gateway --> MsgSvc["Message Ingestion Service"]
MsgSvc --> Kafka["Kafka Topic: chat_events"]

Kafka --> MsgStoreWorker["Message Store Worker"]
MsgStoreWorker --> Cassandra["Cassandra Cluster (Messages DB)"]
(Partition: chat_id)

Kafka --> PushRouter["Push / Route Worker"]
PushRouter --> SessionSvc
PushRouter -->|If Receiver Online| WS_Gateway_B["Receiver's WS Gateway"]
WS_Gateway_B --> ClientB["Receiver Client B"]

PushRouter -->|If Receiver Offline| APNS["APNs / FCM (Push Notification Gateway)"]
  
```

4. Database Schema Design (Apache Cassandra)

Cassandra is selected because it excels at ultra-high write throughput (LSM-Tree) and sequential range queries by time:

```

CREATE TABLE messages (
    chat_id uuid,
    message_id timeuuid, -- Guarantees time-ordered monotonic sorting
    sender_id uuid,
    content text,
    media_url text,
    status text, -- SENT, DELIVERED, READ
    created_at timestamp,
    PRIMARY KEY ((chat_id), message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);
  
```

5. Deep-Dive: Delivery Guarantees & Sync

```
sequenceDiagram
    autonumber
    actor A as User A
    participant Svr as Message Server
    participant DB as Cassandra
    actor B as User B

    A->>Svr: send_message(payload, client_msg_id)
    Svr->>DB: persist_message(status = SENT)
    Svr->>A: ACK: status = SENT (Single checkmark)

    alt User B Online
        Svr->>B: deliver_message(payload)
        B->>Svr: ACK: delivered
        Svr->>DB: update status = DELIVERED
        Svr->>A: notification: status = DELIVERED (Double checkmarks)
    else User B Offline
        Note over Svr: Store in pending queue; trigger Push Notification
    end
end
```